

9. אסטרטגיית הפתרון הנבחרת

9.1 סקירה כללית

האסטרטגיה הנבחרת לפרויקט זה היא אלגוריתם חימוש וסריקה מתקדמת מסוג **חיפוש קרן מיטבי מודע-שלבים (Phase-Aware Best-First Beam Search)**. שיטה זו משתייכת למשפחת האלגוריתמים מוכווני-המידע (Informed Search) לניתוח ובחירת מהלכים על גבי גרף מצבים, כפי שנסקרה בסעיף 8. המערכת מייצגת את המשחק כאוטומט של גרף-מצבים מכוון וללא מעגלים (DAG), שבו כל צומת הוא קונפיגורציית לוח חוקית, וכל קשת היא מהלך זריעה חוקי.

כדי להתגבר על הפיצוץ האקספוננציאלי של מרחב המצבים מבלי להפר את הגבלות הנוהל, המנוע פועל באמצעות ארכיטקטורה מובנית של ארבע שכבות עבודה משולבות:

1. **אוטומט שלבי המשחק (FSA)**: מסווג את עמדת הלוח לאחד מחמישה שלבים אסטרטגיים שונים (סעיף 9.3).
2. **פונקציית הערכה היוריסטית דינמית**: מתרגמת את מצב הלוח לציון מספרי משוקלל, כאשר המשקולות משתנות דינמית בהתאם לשלב שזיהה האוטומט (סעיף 9.4).
3. **מנוע חיפוש Best-First Beam Search**: מנהל את חזית החיפוש ומבצע חיתוך רוחב (Pruning) אגרסיבי כדי לשמור על יעילות משאבים (סעיף 9.5).
4. **טבלת טרנספוזיציות (Transposition Table)**: מאחדת מסלולי חישוב זהים ומונעת הערכות היוריסטיות כפולות (סעיף 9.6).

עבור דרגת הקושי הגבוהה ביותר (**Hard**), המערכת מוסיפה על גבי ארבע השכבות הללו שני מנגנונים ייחודיים ובלעדיים: **רכיב שומר-הגנה (Capture Guard)** למניעת הפקרת אבנים (סעיף 9.8), ו**פותר סיום מדויק (Endgame Solver)** לחישוב מתמטי מוחלט של מהלכי ההכרעה (סעיף 9.9).

תהליך קבלת ההחלטות הכללי של הסוכן בתורו מתבצע בסדר הבא:

- במידה ודרגת הקושי היא **Hard**, והלוח נכנס לשלב סיום (פחות מ-12 אבנים בבורות), מופעל מיידית פותר הסיום הממחה. אם נמצא פתרון מתמטי, המהלך המנצח מוחזר ישירות והתהליך מסתיים.
- בכל תרחיש אחר, מופעל מנוע החיפוש המקורב. המנוע מקצה תקציב מצבים שווה לכל אחד ממהלכי השורש החוקיים, מריץ את חישוב הקרן ומפעיל מעבר האצלת ערכים לאחור (Backup) כדי לקבוע את הציון של כל מסלול.
- במידה ודרגת הקושי היא **Hard**, המהלך המנבחר עובר סינון נוסף דרך רכיב ה-Capture Guard. אם מתגלה שהמהלך חושף את הסוכן לחטיפה הרסנית

מצד היריב, המנגנון מבצע עקיפה (Override) ומחזיר מהלך אלטרנטיבי המגן על הלוח.

הפסאודו-קוד הפורמלי והמלא של זרימת האלגוריתם מתועד בסעיף 14.

9.2 מודל היריב: חמדן, לא מושלם

לב האסטרטגיה מבוסס על האופן שבו הסוכן ממדל את התנהגות היריב. אלגוריתם המינימקס הקלאסי (סעיף 8.1) מניח "יריב מושלם" (Perfect Opponent), כלומר שחקן בעל משאבי חישוב אינסופיים המחשב את העץ עד לסוף המשחק ובוחר תמיד במהלך שממקסם את הפסדנו. גישה זו מנותקת מהמציאות של משחק מול שחקן אנושי ומנוגדת להנחיות.

פרויקט זה נוקט בגישה שונה וממדל את היריב כשחקן חמדן מקומי (Adversarial Greedy). המודל מניח כי השחקן שמולנו פועל מתוך אנוכיות וחמדנות קצרת-רואי: הוא אינו סורק ענפי חישוב עמוקים, אלא יבחר בכל תור במהלך המעניק לו את הרווח הגבוה והמיידי ביותר על גבי הלוח באותו הרגע (כגון העדפת מהלך המבצע חטיפה או כזה המעניק תור נוסף).

בחירה זו מוגנת ומנומקת בשני מישורים:

- **ריאליזם הנדסי:** שחקן אנושי ממוצע אינו מסוגל לחשב בראשו מיליוני מצבים קדימה בזמן אמת. מודל יריב חמדן פשוט מייצג בצורה מדויקת בהרבה את תגובות הנגד של המשתמש, ובכך מאפשר לסוכן לחזות מלכודות אנושיות בצורה יעילה.
- **תאימות מלאה להנחיות הנוהל:** הסרת שכבת המינימזציה (שכבת ה- $\min$ בעץ החיפוש) מוציאה את האלגוריתם לחלוטין מגדר המינימקס האסור (סעיף 13 בנוהל), שכן עץ החיפוש אינו מחליף שכבות של $\max$ ו- $\min$ אלא מבוסס על פריסה חד-צדדית המשולבת עם חיזוי מסלול יחיד של תגובה חמדנית.

מודל זה אינו רכיב חדש, אלא מהווה הבשלה ומימוש מלא של אלגוריתם ה-"Adversarial Greedy Search" שהובטח ואושר במסמך הצעת הפרויקט המקורית. המודל מוטמע כמתודה פנימית מוגנת בקוד (`BeamSearch.SimulateGreedyOpponentMove`). בכל פעם שסריקת המנוע מגיעה לצומת המייצג את תורו של היריב, המערכת אינה מפצלת את העץ לכל הכיוונים, אלא מפעילה את מודל היריב ומסמלצת מהלך יחיד נבחר המייצג את תגובתו הריאליסטית, וממנו ממשיכה את החיפוש קדימה. האסטרטגיה הנוכחית אינה מחליפה את הצעת המקור, אלא בונה מעטפת חיפוש חכמה סביבה.

9.3 שכבה 1: אוטומט שלבי המשחק (FSA)

אוטומט מצבים סופי (Finite-State Automaton) מבוסס על הטיפוס `enum` `GamePhase` ומשמש כשכבה הראשונה של המנוע. האוטומט מנתח את מאפייני פיזור האבנים על גבי הלוח ומסווג כל עמדה לאחד מחמישה שלבים אסטרטגיים קבועים:

משמעות אסטרטגית ופעולה	תנאי כניסה חישובי	שלב (State)
שלב פתיחת המשחק. המגרש עמוס באבנים, והדגש מושם על תפיסת בורות מפתח וייצור לאיצנטופ חטיפות עתידי.	סך האבנים ב-12 הורות ≥ 36	Opening
שלב אמצע המשחק. ניהול איזון עדין בין צבירת נקודות במאגר, שמירה על טילוש בורות, ומניעת איומי יריב.	סך האבנים ב-12 הורות בין 12 ל-36	Midgame
שלב הסיום הפיזי. האבנים על הלוח מועטות, המשחק הופך טקטי והמשקל עובר להכרעה מוחלטת של הניקוד.	סך האבנים ב-12 הורות > 12	Endgame
שלב הרעבה. היריב חנוק ובקושי מחזיק באבנים לתמרון. המטרה היא למנוע הזנת אבנים לצד שלו כדי לאלץ אותו לסיים את המשחק.	סך האבנים בבורות היריב ≤ 3	Starvation Mode
שלב שרשרת חטיפות. עמדת הלוח רגישה ומכילה הזדמנויות חטיפה מרובות זמינות. המשקל עובר למיקסום ותעדוף חטיפות.	קיום ≥ 2 איומי חטיפה פעילים	CaptureChain

האוטומט הוא חסר-זיכרון (Stateless) ומחשב את מצבו מחדש בכל פנייה בזמן קבוע של $O(1)$ על ידי סריקה חסומה של 14 התאים, ללא שמירת מצב בין מהלכים. במידה ועמדת לוח מסוימת מקיימת את תנאי הכניסה של יותר משלב אחד במקביל, האוטומט

מכריע את הסיווג על פי סדר קדימויות קשיח המייצג את הדחיפות האסטרטגית של העמדה:

$\text{StarvationMode} \succ \text{CaptureChain} \succ \text{Endgame} \succ \text{Midgame} \succ \text{Opening}$

סיווג דינמי זה הופך את פונקציית ההערכה ההיוריסטית לפונקציה חלקית (Piecewise Function) המותאמת במדויק לקצב ומצב המשחק, במקום להישען על נוסחה סטטית אחת קשיחה לכל אורך הדרך.

9.4 שכבה 2: פונקציית ההערכה ההיוריסטית הדינמית

נוסחת ההערכה ההיוריסטית (h) מתרגמת את הצומת הנסרק למספר ממשי יחיד. ככל שהציון גבוה יותר, המצב נחשב בטוח ומבטיח יותר עבור הסוכן הממוחשב. הנוסחה מוגדרת כך:

$$h(\text{state}) = w_1 \cdot \Delta S + w_2 \cdot (\text{myCapture} - \text{oppCapture}) + w_3 \cdot D + \text{ChainBonus} - \text{StarvationPenalty}$$

מרכיבי ליבת הנוסחה:

- **ΔS (הפרש הניקוד הממומש):** מספר האבנים במאגר הסוכן פחות מספר האבנים במאגר היריב. זהו המדד המרכזי לניצחון.
- **$\text{myCapture} - \text{oppCapture}$ (פוטנציאל חטיפות נטו):** חישוב אסימטרי משוקלל של איומי החטיפה הקיימים על הלוח. המערכת מעניקה משקל גבוה (ציון 2.0) לאיומי חטיפה אקטיביים הניתנים למימוש מיידי בתור הנוכחי, ומשקל נמוך (ציון 0.5) לאיומים פסיביים הדורשים מהלך הכנה מוקדם. איומי חטיפה של היריב מבוצעים בסימן שלילי כדי לקזז את הציון.
- **D (שליטה בלוח):** סך כל האבנים המונחות בבורות הרגילים של הסוכן. משמש כ"תחמושת" חיונית ליכולת תמרון וניהול מהלכים עתידיים.

דרישת "התולקשות הדינמיות" מיושמת על ידי כך שכל שלב משחק שהוגדר ב-FSA טוען וקטור משקולות (w_1, w_2, w_3) שונה לחלוטין, המשנה את אופי ההתנהגות של הסוכן:

רכיבים מיוחדים משולבים	שדה משקולות w_3 (טילוש)	שדה משקולות w_2 (חטיפה)	שדה משקולות w_1 (ניקוד)	שלב משחק פעיל

Opening	0.4	0.7	1.0	—
Midgame	0.3	0.6	1.0	—
Endgame	1.5	0.4	0.1	משקל הניקוד עולה ל-1.5 (כל אבן במאגר קריטית). משקל הטילוש צונח ל-0.1.
StarvationMode	0.6	1.0	0.2-	משקל הטילוש הופך לשלילי (-0.2); אגירת אבנים בצד שלי מאריכה את חיי היריב ונענשת.
CaptureChain	0.3	1.2	1.0	משקל החטיפה עולה ל-1.2 ומופעל רכיב ה-ChainBonus.

רכיבים מיוחדים מותנים:

- `\text{ChainBonus}` (בונוס שרשרת): מופעל בלעדית בשלב ה-CaptureChain ומעניק תוספת של 1.5 נקודות לכל מהלך המייצר רצף של איומי חטיפה עוקבים, כדי לעודד את הסוכן לגרור את המשחק למערכה אגרסיבית.
- `\text{StarvationPenalty}` (קנס הרעבה): מופעל (בכל השלבים) כאשר ליריב נותרות 3 אבנים או פחות בבורות שלו. הרכיב מטיל קנס קבוע של 3.0 נקודות עבור כל אבן של היריב, כדי למנוע מהסוכן לבצע מהלכים המזינים אבנים לצד של היריב ובכך "מצילים" אותו מהחנק האסטרטגי.

קיצור דרך טרמינלי: כאשר מנוע ההערכה מזהה מצב סיום משחק רשמי (`isGameOver == true`), הנוסחה ההיוריסטית מנוטרלת לחלוטין. המתודה מפעילה את פונקציית הסיום, מחשבת את הפרש האבנים האמיתי והסופי בין המאגרים, ומכפילה אותו בגורם קשיח ענקי של 10,000. צעד זה מבטיח כי עמדות המובילות

לניצחון מוחלט יועדפו תמיד על פני כל עמדה היוריסטית משוערת אחרת, ומצבי הפסד יידחו לחלוטין.

9.5 שכבה 3: מנוע החיפוש Best-First Beam Search

מנוע החיפוש אחראי על פריסת המהלכים וסריקת מרחב המצבים בצורה מבוקרת. המנוע מבוסס על העקרונות המבניים הבאים:

- **ניהול חזית החיפוש (Frontier):** מנוהל באמצעות מבנה נתונים עצמי מסוג ערימת מינימום (MinHeap). הצמתים מוכנסים לערימה עם עדיפות שלילית (h-), תכונה המבטיחה כי הצומת בעל הציון ההיוריסטי הגבוה ביותר צף תמיד לראש המערך ונשלף ראשון ($O(\log n)$), ובכך מייצר התנהגות של חיפוש מכוון מידע (Best-First).
- **חלוקת תקציב שוויונית בין מהלכי השורש:** המנוע קולט את תקציב הצמתים הכולל המוקצה לדרגת הקושי (לדוגמה, 500 מצבים ברמת Medium) ומחלק אותו בצורה שווה לחלוטין בין כל מהלכי השורש החוקיים הזמינים לסוכן באותו התור. חלוקה זו מבטיחה כי כל תת-עץ של מהלך פתיחה ייחקר לאותו עומק ובאותה רמת השקעה משאבית, דבר המאפשר לבצע השוואה הוגנת ומדויקת ביניהם בשלב ה-Backup ללא עיוותים הנגרמים מחקירה לא שוויונית.
- **חיתוך רוחב (Beam Pruning — פריסה אסימטרית):** זרימת החיפוש מפרידה בין צעדי הסוכן לצעדי היריב:
 - **בתורו של הסוכן:** המנוע מייצר את כל המהלכים החוקיים, מדרג אותם ומותיר בחזית החיפוש אך ורק את K הצמתים המובילים בעלי הציון ההיוריסטי הגבוה ביותר (רוחב הקרן), בעוד ששאר הענפים נחתכים ונזרקים מיידית.
 - **בתורו של היריב:** המנוע אינו מפצל את העץ לכל הכיוונים, אלא מפעיל את מודל היריב החמדן ומייצר צומת המשך יחיד (Single-child path) המייצר את מסלול החיזוי הישיר של תגובתו הצפויה.
- **כיווץ שרשרת תורות נוספים:** חוק "תור נוסף" במנקלה מאפשר לבצע שרשרת מהלכים באותו התור. המנוע מזהה מצבים אלו ומריץ לולאת פיזור פנימית המכווצת את כל רצף המהלכים הללו לכדי קשת אחת ארוכה ומאוחדת בגרף המצבים. עבור האוטומט, צעד אחד קדימה מייצג תמיד "תור שלם" שהסתיים והועבר לצד השני, ולא מהלך פיזור בודד.
- **אופק עומק קשיח:** החיפוש מוגדר עם חסם עומק מקסימלי (על פי דרגת הקושי הפעילה). צומת שהגיע לאופק העומק אינו מורחב יותר לחזית, אלא מוגדר כעלה ומקבל את הציון ההיוריסטי המידי שלו, שבו נשמר מודל היריב החמדן בצורה המהימנה ביותר.

החיפוש מסתיים בצורה דטרמיניסטית ברגע שתקציב הצמתים של תת-העץ נוצל במלואו או כאשר חזית החיפוש התרוקנה.

9.6 שכבה 4: טבלת הטרנספוזיציות

כדי למנוע בזבוז משאבי עיבוד יקרים על הערכות היוריסטיקות חוזרות של מצבי לוח זהים שהושגו בסדר מהלכים שונה, המנוע מפעיל טבלת גיבוב מובנית מסוג [Dictionary<long, double>](#).

הערך המשמש כמפתח מחושב באמצעות אלגוריתם הגיבוב המהיר **FNV-1a** **(64-bit)** המריץ פונקציית כפל ו-XOR מתמטית רציפה על גבי מערך 14 התאים של הלוח, בשילוב זהות השחקן התורן. ערך זה מיוצר בזמן קבוע של $O(1)$. פעולת הגיבוב המתמטית משיגה בדיוק את אותו פיזור ואותו יעד הנדסי שהובטח בהצעת הפרויקט תחת השם Zobrist Hashing (איחוד מצבי לוח זהים בגרף ומניעת כפילויות), אך היא חסכונית ומבוזרת בהרבה, שכן היא אינה דורשת הקצאת זיכרון מוקדמת ותקורה של מערכי מספרים אקראיים ענקיים על גבי לוח ביטים.

הטבלה "מקפלת" את מבנה עץ החיפוש הרגיל והופכת אותו לגרף מכוון ללא-מעגלים (**DAG**), המבטיח שכל עמדה ייחודית תחושב פעם אחת בלבד.

9.7 מהלך בחירת המהלך: שלב ה-Backup

בסיום פעולת החיפוש ופריסת הצמתים, מופעל מעבר האצלת ערכים לאחור (**Backup**) על גבי עץ החיפוש שנסרק. המעבר מבוצע באמצעות מתודה רקורסיבית הנעה מהעלים חלפי מעלה אל עבר צמתי השורש, ומחשבת לכל צומת את ערכו הריאליסטי בהתאם לכללי מודל המשחק:

- **בצומת עלה (צומת שלא הורחב או שהגיע לאופק):** הערך המואצל הוא הציון ההיוריסטי הישיר של המצב ($Value = h$).
- **בצומת השייך לסוכן (שכבת MAX מקומית):** הערך המואצל הוא הערך המקסימלי מבין כל ערכי ה-Backup של צאצאיו שנסרקו בקרן ($Value = \max(Children)$). הסוכן מניח כי בתורו הוא יבחר במסלול המבטיח לו את הציון הגבוה ביותר.
- **בצומת השייך ליריב (שכבת מינימלית):** הצומת מכיל צאצא יחיד המייצר את מסלול התגובה החמדנית שנחזתה במודל. הערך המואצל של הצומת שווה במדויק לערך ה-Backup של אותו ילד יחיד, ללא ביצוע פעולת מינימיזציה.

המהלך הנבחר שיוחזר ויבוצע על הלוח הראשי הוא מהלך השורש שקיבל את ערך ה-Backup הגבוה ביותר בתום המעבר. מנגנון זה מחליף את הבחירה הנאיבית של "הצומת הטוב ביותר שנראה אי פעם בעץ", ומבטיח כי המהלך שנבחר מוביל למסלול

יציב ומאזן לאורך זמן, תוך דחיית מסלולים המכילים "פנטזיות" רגעיות שביניהן ממוקמים מהלכי כשל הרסניים.

9.8 מנגנון שומר-ההגנה (Capture Guard — בלעדי ל-Hard)

מנוע החיפוש המקורב חסום בעומק וברוחב הקרן, ועלול במצבים מסוימים ליפול ל"אפקט האופק" — מצב שבו מהלך נראה רווחי בטווח המיידי של העץ, אך הוא מפקיר בור ריק המאפשר ליריב לבצע חטיפה הרסנית מיד בצעד הבא שאחרי האופק הנסרק.

על מנת לספק רשת ביטחון מוחלטת, רמת הקושי **Hard** מפעילה שכבת סינון קשיחה עליונה (**ApplyCaptureGuard**) מיד לאחר בחירת המהלך באלגוריתם:

- **בדיקת חשיפה:** המנגנון מריץ סימולציה מהירה של המהלך שנבחר ובודק את תגובת הנגד המיידית של היריב האנושי בתור הבא.
- **תנאי הפסילה:** אם מתגלה כי המהלך שנבחר משאיר ליריב הזדמנות לבצע חטיפה משמעותית (חטיפה של 3 אבנים או יותר בחינם) — והסוכן עצמו אינו מבצע חטיפה נגדית גדולה באותו המהלך — המהלך מוגדר כ"מהלך כשל" ונפסל מיידית.
- **עקיפה והגנה (Override):** המנגנון סורק את שאר מהלכי השורש החוקיים, מוצא את המהלך האלטרנטיבי המעניק את רמת החשיפה המינימלית ביותר ליריב (המהלך המגן על הלוח), ומחזיר אותו לביצוע במקום המהלך המקורי שנפסל.

רכיב ה-Capture Guard מהווה חוק מתמטי קשיח המגן על הסוכן מפני פנטזיות אסטרטגיות ומבטל לחלוטין את הבלנדרים הלא-מגנים שנצפו בעמדות הבדיקה (כמו ב-pic4), ובכך הופך את רמת ה-Hard ליריב חסין, עקבי ומאתגר.

9.9 פותר מצבי הסיום (Endgame Solver — בלעדי ל-Hard)

השכבה החמישית והעליונה של המערכת, הפועלת בלעדית ברמת קושי Hard, היא פותר הסיום הממצה (**EndgameSolver**). כאשר כמות האבנים הכוללת ב-12 הבורות הרגילים יורדת מתחת ל-12 אבנים, מרחב המצבים של המשחק מצטמצם בצורה משמעותית והמשחק הופך טקטי לחלוטין. ברגע זה, המנוע מפעיל את פותר הסיום ומדלג על חישוב ה-Beam Search המקורב:

- **סריקה ממצה וערכים מוחלטים:** הפותר מריץ סריקה רקורסיבית ממצה המגיעה עד לעלי הסיום המוחלטים של המשחק (**IsGameOver == true**). ברמת העלים מופעלת פעולת ה-**FinalizeBoard** ומחושב הפרש הניקוד הסופי האמיתי שיושג בלוח. ערך זה אינו הערכה היוריסטית משוערת אלא מספר מתמטי מוחלט וודאי.

- **ארכיטקטורה חד-צדדית דטרמיניסטית:** הפותר שומר על חוקי המודל ומבצע בחירת מקסימום ($\max$) על מהלכי הסוכן, בשילוב חיזוי מהלך חמדני יחיד של היריב (`SimulateGreedyOpponentMove`), ללא פריסת שכבת $\min$ מלאה. תכונה זו שומרת עליו מחוץ לגדר אלגוריתם המינימקס האסור בנוהל.
- **טבלת זיכרון חסינת התנגשויות (Memoization):** הפותר מנהל טבלת זיכרון ייעודית מסוג `<Dictionary<string, int>` המבוססת על מחרוזת מלאה של 14 התאים והתור הנוכחי (סעיף 12.7). הטבלה מונעת חישובים כפולים ומאחדת את עץ הסיום לגרף DAG מוחלט.
- **תקציב בטיחות:** כדי למנוע תקיעה של הממשק במצבי קצה מורכבים, נקבע תקציב בטיחות קשיח של 250,000 מצבים חלוטין. במידה והעץ חורג מהתקציב, המנגנון מבצע נסיגה (Fallback) חנינית ומעביר את ניהול המהלך ל-Beam Search הרגיל.

שילוב פותר הסיום מעניק לרמת ה-Hard יתרון עוצמתי מוחלט — היא פשוט אינה מסוגלת לבצע טעויות חישוב בשלב סיום המשחק ומנווטת את האבנים להכרעה מתמטית מושלמת.

9.10 פרמטרי שלוש רמות הקושי

שלוש רמות הקושי מפעילות את אותו מנוע אלגוריתמי בסיסי, אך נבדלות ומובחנות ביניהן באמצעות שלושה פרמטרים מבניים קשיחים השולטים על עומק ורוחב הסריקה:

רמת קושי	אופק עומק החיפוש (Depth) (Horizon)	תקציב צמתים כולל (Node Budget)	רוחב הקרן החותר (K)	שכבות הגנה משולבות (Guard / Solver)
Easy	plies 2 (מהלכים קדימה)	100 צמתים	4 מצבים מובילים	ללא
Medium	plies 3 (מהלכים קדימה)	500 צמתים	8 מצבים מובילים	ללא

מנגנון Capture Guard + פותר סיום Endgame Solver	8 מצבים מובילים	2,000 צמתים	plies 4 (מהלכים קדימה)	Hard
---	-----------------	-------------	-------------------------------	-------------

אכיפת דטרמיניזם מלא: במטרה לעמוד במדויק בסעיף 7 בנוהל הרשמי, מנוע החיפוש מנותק לחלוטין מרכיבי אקראיות או משעון המערכת (**DateTime**). עצירת החיפוש מבוקרת בצורה דטרמיניסטית קשיחה אך ורק על ידי ניצול תקציב הצמתים והגעה לאופק העומק המספרי. כתוצאה מכך, עבור אותה עמדת לוח נתונה, הסוכן יפיק תמיד את אותו המהלך בדיוק, ללא תלות בעומס המעבד או בחומרה עליה מורץ המשחק. דרגת הקושי נקבעת ומבוקרת מדעית אך ורק על ידי שינוי אופק הסריקה ועומק הראייה, ללא שימוש בשער אקראי פסול.

9.11 מדוע האלגוריתם אינו מינימקס ועומד בהנחיות הנוהל?

סעיף 13 בנוהל אוסר במפורש שימוש באלגוריתמים הקלאסיים הממצים המוכרים: Minimax, Alpha-Beta Pruning, ו-MCTS. האסטרטגיה הנבחרת בפרויקט זה נבדלת מהם מבנית ומתמטית על פי הקריטריונים הבאים:

1. **היעדר שכבת מינימיזציה (No MIN Layer):** אלגוריתם מינימקס מפצל את העץ בתורו של היריב ובודק את כל המהלכים החוקיים שלו מתוך הנחה שהוא יבחר במהלך בעל הציון הנמוך ביותר עבורנו (**min**). האלגוריתם הנוכחי אינו מבצע פיצול או מינימיזציה בתורו של היריב; הוא מחשב צעד יחיד חמדני צפוי מתוך מודל היריב, ובכך הוא מהווה פריסה חד-צדדית מוכוונת מסלול ואינו נמנה עם משפחת המינימקס.
2. **האצלת ערכים חד-צדדית (Asymmetric Backup):** בשלב ה-**Backup**, המערכת אינה מחליפה פונקציות של **max** ו-**min** בין השכבות. בצמתי היריב, הערך המועלה מעלה שווה במדויק לערך הצאצא היחיד שנחזה במסלול, ללא הפעלת פונקציית מינימום.
3. **ניהול חזית מבוסס תור עדיפויות:** המינימקס סורק את המרחב בצורה עיוורת לפי סדר עומק (DFS) תוך שימוש במחסנית קריאות רקורסיבית מוחלטת. האלגוריתם הנוכחי מנהל את חזית החיפוש בצורה רוחבית מוכוונת מידע באמצעות תור עדיפויות מנוהל (**MinHeap**).
4. **משקולות דינמיות מבוססות אוטומט:** נוסחת ההערכה במינימקס קלאסי היא קשיחה וקבועה. המערכת הנוכחית משנה ומחליפה את משקולות הנוסחה דינמית בזמן ריצה באמצעות מודל מכונת מצבים (FSA).

5. **עצירה מבוססת תקציב משאבים:** החיפוש עוצר ומבוקר על ידי ניצול תקציב צמתים נוקשה, בניגוד לסריקה הממצה עד אופק מלא המאפיינת את המינימקס והאלפא-בטא.

6. **חוסר אקראיות מוחלט:** בניגוד לאלגוריתם ה-MCTS הנשען על סימולציות סטטיסטיות אקראיות (Playouts) ושעון זמן, הליבה הנוכחית היא דטרמיניסטית ומתמטית לחלוטין ומהווה פונקציה טהורה של מצב הלוח.

גם פותר מצבי הסיום (**EndgameSolver**), למרות שהוא סורק את המרחב עד לעלי הסיום, אינו משתמש בשכבת **min** מלאה או בחישוב מינימקס קלאסי; הוא שומר על אותה ארכיטקטורה חד-צדדית המנבאת מהלך חמדני בודד של היריב, ולכן הוא חוקי לחלוטין ועומד במלואו בהנחיות החובה של הנוהל.